

Prediction Algorithm Accuracy Report — v3

Multi-Season Data: What More Data Actually Bought Us

Sports Analytics Project

September 2026

Abstract

This report covers the third round of work on the platform’s prediction algorithms, following two single-season reports (`prediction-accuracy-report.pdf`, `-v2.pdf`, same directory) that concluded recency-weighted feature recombination was exhausted and the two highest-value remaining bets were a genuine data-correctness fix and more historical data. Both were executed: a real trade-attribution bug was found and fixed (players’ historical box scores were being attributed to their *current* team, not the team they played for in that specific game), and two additional complete NBA seasons were ingested, taking the dataset from 1,321 games to 3,781. Re-running the Elo constant grid search against the enlarged dataset produced a different, validated result from the single-season attempt: tuned constants (`HOME_COURT_ADVANTAGE_ELO` = 40, `K_FACTOR` = 25) now beat the literature defaults on held-out data, confirmed stable across a wide parameter grid and two independent validation splits, and have been shipped. Every other model improved as well, most strikingly the two recency-weighted player-point predictors, whose edge over a naive baseline roughly doubled. We also document two engineering incidents from the ingestion process itself — a Postgres connection pooler killing idle/long-lived connections far more aggressively than expected — since the fixes are reusable for any future data pull.

Contents

1	What Changed Since v2	2
2	The Trade-Attribution Bug	2
3	The Multi-Season Elo Re-Tuning: A Different Result This Time	3
3.1	What changed	3
3.2	What shipped	3
3.3	A caveat we deliberately did not address	4
4	Impact on the Other Three Models	4
5	Engineering Note: Pooled-Connection Fragility During Ingestion	5
6	Recommendations Going Forward	5

Change	Status	Detail
Trade-attribution bug fix	Shipped	New <code>PlayerGameStat.teamId</code> column, schema migration, ingestion fix, backfill of 33,590 existing rows, <code>four_factors.py</code> query fix (§2)
2023-24 season ingested	Shipped	1,230 games, full boxscores, correct team attribution from day one
2024-25 season ingested	Shipped	1,230 games, same
Elo constants re-tuned	Shipped	<code>HOME_COURT_ADVANTAGE_ELO</code> 75 → 40, <code>K_FACTOR</code> 20 → 25 — this time validated, unlike the v1/v2 attempt (§3)
Ingestion connection resilience	Shipped	<code>ingest_historical_season.py</code> — fresh DB connection per game write, not one held across the whole run (§5)

Table 1: Disposition of this round’s changes.

1 What Changed Since v2

2 The Trade-Attribution Bug

While scoping the multi-season pull, we found that `four_factors.py`’s team-game boxscore aggregation joined `PlayerGameStat` to `Team` via `Player.teamId` — a player’s *current* team — rather than the team they actually suited up for in a specific historical game. A player traded mid-season had every one of their pre-trade games silently reattributed to their new team the moment the roster ingestion next ran. An audit query confirmed this was not theoretical: **7.7% of the 33,590 `PlayerGameStat` rows** already had a `Player.teamId` that matched neither team in that row’s own game.

The fix required a real schema change, not just a query edit, since the correct information (which team each boxscore row’s player was on, for that specific game) was being read from the NBA API and then discarded before ever reaching the database:

1. Added `PlayerGameStat.teamId` via a Prisma migration (nullable, foreign-keyed to `Team`).
2. Updated `apps/ingestion/games.py/ingest.py` to capture and write this field going forward — the boxscore endpoint already reports it per player, it just wasn’t being kept.
3. Backfilled all 33,590 existing rows: 31,012 were unambiguous (the player’s current team matched one of the two teams in that game, so no fix was needed) and were updated directly; the remaining 2,578 rows (774 distinct games) needed a boxscore re-fetch to resolve correctly, since the original ingestion never stored per-game team identity.
4. Updated `four_factors.py`’s query to join on the new field.

Measured impact, isolated from the data-volume increase (re-run against the original single-season dataset before any new games were added): MAE improved from 12.52 to 12.51 points, walk-forward MAE from 12.65 to 12.62. A small, real improvement — expected, since only 7.7% of rows were affected and the fix redistributes existing signal between two teams’ season averages

rather than adding new information. The fix was worth making on correctness grounds regardless of its accuracy impact, and matters increasingly as more seasons (with more real trades) are added.

3 The Multi-Season Elo Re-Tuning: A Different Result This Time

3.1 What changed

v1/v2 reported that a chronological train/validation grid search over `HOME_COURT_ADVANTAGE_ELO` and `K_FACTOR`, run against the single-season dataset (1,321 games), picked a training-split winner that validated *worse* than the FiveThirtyEight literature defaults ($HCA=75$, $K=20$) — textbook overfitting, not a real improvement, attributed to too little independent evidence in one season to beat a well-chosen prior.

With the dataset now at 3,781 games (2023-24, 2024-25, 2025-26), the identical grid search was re-run.

Split	Default ($HCA=75$, $K=20$) Brier	Tuned ($HCA=40$, $K=25$) Brier
70/30 chronological split	0.2142	0.2119
60/40 chronological split	0.2123	0.2101

Table 2: Validation-split Brier score, default vs. tuned constants, multi-season dataset. Lower is better; tuned constants win on both independent splits this time.

Two checks were run before trusting this, given the exact same experiment produced the opposite conclusion on less data:

- **Grid stability.** The winning pair ($HCA=40$, $K=25$) was re-confirmed against a much wider grid ($HCA \in [0, 110]$, $K \in [5, 45]$) and sits comfortably in the interior, not pinned against either boundary — a boundary-pinned winner would suggest the true optimum lies outside the tested range and the result shouldn't be trusted as-is.
- **Split independence.** The same winner was reproduced from two different chronological train/validation cut points (70/30 and 60/40 of the season timeline), not just the one split used in the original single-season report. Both splits favor the tuned constants over the defaults by a similar margin, which is the standard cross-validation argument that this is a real effect, not an artifact of exactly where one particular split happened to fall.

The training-split Brier surface itself also tightened meaningfully with more data — the top-10 grid candidates spanned 0.2233–0.2251 on the single-season dataset (a difference small enough to be mostly noise) versus 0.2156–0.2162 on the 3-season dataset (a narrower, more confidently-identified optimum). This is exactly the signature you'd expect if the earlier result really was a data-volume problem rather than a methodology problem, as the v1/v2 report speculated.

3.2 What shipped

`HOME_COURT_ADVANTAGE_ELO` was changed from 75 to 40, and `K_FACTOR` from 20 to 25, in `elo.py`. `GamePrediction` was regenerated for all 3,781 games under the new constants.

The permutation test (1,000 randomized-outcome reruns of the real schedule) again rejects the null decisively ($p < 0.001$) — the model's accuracy is real signal, not an artifact of the larger dataset

Metric	Single-season (old constants)	Multi-season (tuned constants)
n	1,321	3,781
Brier score	0.2146	0.2145
Accuracy (threshold 0.5)	65.2%	66.0%
Naive home-favorite baseline	55.5%	54.8%

Table 3: Full-dataset accuracy, old vs. new. Brier score is nearly unchanged on the full (non-held-out) figure — the real gain is in calibration (Table 4) and in how the model generalizes to unseen games (the validation-split numbers above), not in a large full-dataset summary-statistic move.

Predicted range	n (old)	Old: pred / actual	n (new)	New: pred / actual
0.2–0.3	27	0.257 / 0.037	311	0.253 / 0.296
0.3–0.4	87	0.357 / 0.230	421	0.352 / 0.363
0.4–0.5	181	0.454 / 0.359	572	0.453 / 0.442

Table 4: Calibration in the range where v1 found the worst overconfidence. The predicted-vs-actual gap (e.g. predicting 0.454 when the true rate was 0.359, a 0.095 overconfidence) has closed substantially at the new constants (0.453 predicted vs. 0.442 actual, a 0.011 gap) — this is the practically meaningful improvement, even though the single headline Brier number barely moved.

alone.

3.3 A caveat we deliberately did not address

`compute_elo_ratings` treats every game across all three seasons as one continuous chronological sequence — a team’s rating carries directly from the last game of one season into the first game of the next, with no reset or regression-toward-the-mean for the roster turnover a real NBA off-season involves (trades, draft, free agency). This is a real simplification. It was deliberately left unaddressed in this round so the re-tuning experiment isolated one variable (more data under the existing model) rather than two (more data *and* a season-boundary model change) — conflating them would have left it unclear which change actually produced the improvement. A season-aware reset is a reasonable candidate for a future, separately-evaluated change.

4 Impact on the Other Three Models

Model	Single-season	Multi-season
Four Factors margin MAE	12.52	12.30
Four Factors naive-baseline MAE	13.36	12.91
Fantasy points: recency-weighted MAE	7.74	8.00
Fantasy points: naive-mean MAE	8.25	8.94
Fantasy points: recency-weighting edge	+0.52	+0.94
Scorer points: recency-weighted MAE	4.45	4.63
Scorer points: naive-mean MAE	4.64	4.98
Scorer points: recency-weighting edge	+0.20	+0.36

Table 5: All four models, single- vs. multi-season. MAE in points (or fantasy points) throughout.

Four Factors improved modestly ($12.52 \rightarrow 12.30$ MAE) — a real gain, consistent with more team-game observations sharpening the same 3-feature regression, but not a transformation the way Elo’s calibration was. It remains the weakest of the four models relative to its naive baseline and should stay a secondary, lower-confidence signal in the product.

Both recency-weighted player-point predictors got measurably better relative to their own naive baseline, even though their raw MAE went up slightly. This is not a contradiction: MAE going up while the *edge over naive* roughly doubles means the naive baseline got substantially worse with more data (a longer unweighted history dilutes toward a player’s career-long average, increasingly stale as more games accumulate), while the recency-weighted model held up better against the same harder task. In other words, recency-weighting’s specific value proposition — tracking current form instead of career-long average — becomes more important, not less, exactly as more historical data is added. This is the strongest positive result in this report for a fully-shipped, no-caveats model.

5 Engineering Note: Pooled-Connection Fragility During Ingestion

Two separate incidents during the historical-season ingestion are worth recording since the underlying cause recurs for any future long-running script against this project’s Supabase database.

Incident 1 — idle connection killed during a slow, mostly-network-bound phase. The original `ingest_historical_season.py` opened one database connection at the start of the script and held it through `collect_season_game_dates`, a phase making 30 rate-limited `LeagueGameFinder` calls with no database activity in between. That connection was killed by Supabase’s pooler before a single write happened, with the error `server closed the connection unexpectedly`.

Incident 2 — connection killed mid-write-loop, even while actively used. After fixing incident 1 by opening a fresh connection only for the write phase, the run still failed, repeatedly, partway through — including within the first 50 games. A direct test isolated the cause precisely: **a connection idle for as little as 5 seconds was already killed** (`cur.execute('SELECT 1')` on a 5-second-idle connection failed with the same error). Each game in the write loop involves a rate-limited ~ 1 second-plus network call to `stats.nba.com` *before* the next database write — comfortably enough idle time, every single game, to exceed a 5-second pooler timeout.

Fix. Rather than reconnect-on-failure (tried first; the pooler was frequent enough that even a retry-once policy failed multiple times within the first 50 games), the script now opens, writes, commits, and closes a fresh connection for *every individual game*. This adds one extra network round-trip per game but means every write is durably committed the instant it happens — a crash or interruption loses at most one in-flight game, and simply re-running the script (every write is an idempotent upsert) picks up exactly where it left off. This pattern generalizes to any future long-running script against this database: do not hold one connection across a loop with rate-limited network calls between writes; treat this pooler’s practical idle budget as a few seconds, not the tens of seconds or minutes that might be assumed for a typical managed Postgres instance.

6 Recommendations Going Forward

1. **Treat the Elo re-tuning result as a positive signal for pulling even more data**, specifically for Four Factors and the two player-point predictors, none of which improved as

dramatically as Elo did. Elo's constants were the most data-starved parameter in the whole system (only 2 free parameters, literature priors already close), so it's the one most likely to show a clean signal early; the others may simply need more than 3 seasons to show a similarly clear multi-season effect.

2. **Revisit the season-boundary reset question** (§3's caveat) as a focused, separately-evaluated follow-up — now that Elo's core constants are validated against real multi-season data, adding a principled offseason regression-toward-the-mean is the next natural refinement, and can be backtested the same way (train/validation split, compare against the current no-reset baseline).
3. **Apply the same trade-attribution correctness discipline going forward:** any future feature that joins boxscore data to a team should use `PlayerGameStat.teamId`, not `Player.teamId` — worth a code-review note or lint rule if more code ends up needing this join.
4. **Reuse the fresh-connection-per-write pattern** (§5) for any future ingestion or long-running write script against this database, rather than rediscovering the 5-second idle timeout the hard way again.

Appendix: Reproducing These Results

`apps/predictor/check_accuracy.py` recomputes the Elo and Four Factors numbers in this report against the database's current state — run it directly rather than trusting the point-in-time figures above once more data has been added. The player-point predictor backtests (recency-weighting edge over a naive running mean) and the Elo grid-search/validation-split methodology are one-off analysis scripts, not part of the committed codebase, reflecting their purpose as a one-time validation exercise rather than ongoing monitoring — `check_accuracy.py` is the durable, re-runnable piece.